



SharpWing

Alma Mater Studiorum – Università di Bologna
Scalable and Reliable Services M
Finance Domain
8 June 2026

Bertozzi Pietro – Foroozanfar Arash – Wewiór Mikołaj
Professor Colajanni Michele

<https://github.com/scalable-reliable-services-agentic-ai/sharp-wing>
<https://github.com/scalable-reliable-services-agentic-ai/report-presentation-demo>

Contents

1	Abstract	3
2	Service Definition	4
2.1	Domain and Sector	4
2.2	Proposed Service	4
2.3	Stakeholders and Users	4
2.3.1	Project Stakeholders	4
2.3.2	System Users	4
2.4	Main Events and Workload	5
2.5	Main Failure Scenarios	5
2.5.1	Infrastructural and Connectivity Failures	5
2.5.2	Operational and Workload Failures	6
3	Requirements and SLOs	7
3.1	Functional Requirements	7
3.2	Non Functional Requirements	7
3.3	Compliance and Governance Requirements	8
3.4	Service Level Objectives (SLO)	8
4	System Architecture	10
4.1	Transactional Data Pipeline	10
4.2	Persistency and Storage	10
4.3	Architectural Trade-Offs Analysis	11
5	Reliability and Scalability	12
5.1	Importance of Scalability and Reliability	12
5.2	Scalability Mechanisms	12
5.3	Reliability Mechanisms	13
6	Observability	14
6.1	Importance of Observability	14
6.2	Observability Mechanisms	14
6.3	Metrics	14
6.3.1	Infrastructure Metrics	14
6.3.2	Application Metrics	15
7	Anomaly Detection	16
7.1	Adversarial Fraud Anomaly Typologies	16
7.2	Architecture: Hybrid Anomaly Detection Engine	17
7.2.1	Deterministic Rule Engine and In-Memory State Tracking	17
7.2.2	Probabilistic Sieve Gates and Machine Learning Inference	17
7.2.3	Self-Bootstrapping and Data Diversity Guard	19
7.2.4	Adaptive MLOps Lifecycle: Champion vs. Challenger Architecture	20
7.2.5	Event-Driven Retraining Loop	20

7.2.6	Architectural Graceful Degradation Framework	21
7.3	Data Persistence and Write-Isolation Framework	21
8	Agentic Operations Layer and Safety	23
8.1	Triage Agent	23
8.1.1	API Rate-Limit Resilience via Exponential Backoff	23
8.1.2	MCP Server Infrastructure and Tool Taxonomy	24
8.2	Judge Agent	26
8.3	Agents Responsibilities	26
8.4	Safety, Policies and Guardrail	27
8.4.1	Primary Triage Agent Invariant Policies	27
8.4.2	High-Net-Worth (VIP) Isolation and Behavioral Volatility Policies . . .	27
8.4.3	Auditor Judge Agent Guardrails (LLM-as-a-Judge)	28
8.4.4	Structural Payload Enforcements	29
9	Threat Model and Security	30
9.1	Network Attacks	30
9.2	Classic Injection Attacks	30
9.3	Modern Injection Attacks	30
9.4	Security Principles and Best Practices	31
10	Cloud Migration	32
10.1	Compliance: DORA, AI Act, GDPR	32
10.2	Multi-Cloud Deployment	33
10.3	Disaster Recovery Plan	33
11	Economic Analysis: ROI and Return on Agent (ROA)	34
11.1	Financial Analysis and Fraud Impact	34
11.2	CAPEX, OPEX, and TCO	34
11.2.1	Capital Expenditure (CAPEX)	34
11.2.2	Operational Expenditure (OPEX)	34
11.2.3	Total Cost of Ownership (TCO)	34
11.3	Return on Investment (ROI)	35
11.4	Return on Agent (ROA)	35
12	Future Developments and Conclusion	36
12.1	Future Developments	36
12.1.1	Strategic Enhancements	36
12.1.2	Evolutionary Necessity	36
12.2	Conclusion	36
13	Bibliography	37

1 Abstract

SharpWing is an internally developed fraud detection system engineered to secure real-time banking transactions against increasingly sophisticated financial threats. Processing millions of transactions daily, the system addresses a critical business challenge: protecting financial assets and customer trust while managing the massive volume and velocity of a modern data-intensive application.

To ensure mission-critical reliability, the platform is orchestrated via Kubernetes and deployed across a multi-cloud infrastructure (AWS and Azure), meeting DORA and AI Act compliance standards.

The core operational difficulty lies in achieving an optimal trade-off between execution speed, infrastructure cost, and analytical precision. SharpWing solves this by leveraging a multi-tiered pipeline that integrates deterministic filtering algorithms, advanced machine learning models, and an autonomous Agentic Operations Layer based on the Model Context Protocol (MCP).

Initial pipeline stages apply low-latency algorithms and machine learning estimators to filter out safe transactions economically. The true innovation lies in the agentic layer: while traditional models provide static risk scores, our specialized Triage Agent actively investigates high-risk anomalies, contextually correlating historical data. Its reasoning is then strictly audited by a Judge Agent, which performs policy validation and ensures that every decision is justified.

Operating under a rigorous Operational Safety Policy, these agents use MCP-exposed tools to manage incident response. In cases of high uncertainty or disagreement between the agents, the system triggers a Human-in-the-Loop (HITL) workflow, allowing analysts to review the explicit reasoning paths for swift, well-informed final decisions.

By dynamically routing data through this hybrid architecture, SharpWing minimizes infrastructure overhead, slashes false positives, and ensures cost-effective fraud mitigation. The resulting Return on Agent (ROA) demonstrates that this agentic integration provides measurable operational value, significantly reducing the alert fatigue of human operators while maintaining a 99.99% service availability.

2 Service Definition

2.1 Domain and Sector

SharpWing operates within the **Financial Services** domain, specifically focusing on **Real-Time Fraud Detection and Operational Triage**. The system addresses the critical need to process massive transactional volumes while maintaining absolute accuracy and near-zero latency in identifying fraudulent patterns.

2.2 Proposed Service

SharpWing is a proprietary, mission-critical platform developed internally to secure the institution's real-time banking operations. As a custom-engineered asset, it is designed for deep integration with internal transactional cores, ensuring absolute data sovereignty and proactive compliance with banking-specific security standards.

The service's core innovation lies in its integration of an **Agentic Operations Layer**. By utilizing the **Model Context Protocol (MCP)** as a hard security boundary, the platform enables autonomous reasoning while strictly confining agent actions within a pre-validated toolset. This architecture ensures that operational automation remains scalable, reliable, safe and auditable.

2.3 Stakeholders and Users

As a proprietary internal platform, **SharpWing** identifies its primary users and stakeholders within the bank's organizational structure. The system is designed to bridge the gap between high-volume technical operations and regulatory compliance requirements.

2.3.1 Project Stakeholders

The stakeholders represent the entities with a direct interest in the system's outcomes and its financial or legal impact:

- **Bank Executive Management:** Interested in protecting the institution's financial assets and maintaining customer trust by minimizing fraud losses. Their focus is on the **Return on Investment (ROI)** and the operational efficiency gained through agentic automation.
- **Compliance and Legal Officers:** Responsible for ensuring that the system adheres to strict regulations such as **DORA**, the **EU AI Act**, and **GDPR**. They require the auditability provided by the *Agents* and the detailed post-mortem reports for every automated decision.

2.3.2 System Users

The users are the technical and operational personnel who interact directly with the platform's interfaces and the agentic layer:

- **SOC (Security Operations Center) Analysts:** The primary operators who act as the **Human-in-the-Loop (HITL)**. They perform deep-dive investigations into suspicious transactions, allowing them to focus on high-complexity cases while the agents handle the initial analysis.
- **SRE (Site Reliability Engineering) Team:** Responsible for the platform's uptime and infrastructure health. They manage the Kubernetes clusters and the multi-cloud deployment strategy, using the system's observability stack to monitor the agents' performance and ensure the **99.99% availability** target is met.

2.4 Main Events and Workload

The primary workload consists of real-time banking transactions arriving through the bank's internal API Gateway. The system is sized according to the following parameters:

- **Daily Baseline:** The system processes a baseline of **20 million transactions per day**.
- **Peak Workload:** During seasonal spikes or high market volatility, the platform is engineered to handle a throughput of **580 RPS**.

To sustain these peaks, the system separates **synchronous ingestion** (p95 latency < 50ms) from **asynchronous processing**. During extreme spikes, **Kafka spike buffering** prevents data loss by queuing incoming transactions, providing the Horizontal Pod Autoscaler (HPA) the necessary time to provision additional replicas for the microservices.

2.5 Main Failure Scenarios

Given its mission-critical role in the bank's core transactional flow, **SharpWing** must proactively identify and mitigate diverse failure modes to ensure institutional stability and data integrity.

Following the fundamental engineering principle that "real systems fail," the platform is designed to maintain acceptable behavior even when individual components or autonomous agents degrade, prioritizing safe fallbacks and human-in-the-loop escalation over speculative automation.

2.5.1 Infrastructural and Connectivity Failures

The primary risks to the system's availability stem from its distributed, multi-cloud nature:

- **Cross-Cloud Network Partitioning:** High-latency spikes or total loss of connectivity between AWS and Azure regions, impacting the real-time synchronization of the fraud state.
- **Managed Service Unavailability:** Failure of critical shared components such as the primary database or one of the caches. The system mitigates this by allowing the ingestion API to continue in a degraded "fail-fast" mode, ensuring no transaction is lost while the data layer recovers.

2.5.2 Operational and Workload Failures

High-throughput periods (580 RPS) introduce specific operational bottlenecks:

- **Kafka Queue Congestion:** During extreme bursts, the asynchronous processing pipeline may exceed its latency SLO. The system utilizes **Kafka spike buffering** to protect the scoring engine from crashing, prioritizing data integrity over immediate response time.
- **Pod Crash Loops:** Failures in the containerized microservices due to resource starvation or software bugs. *Kubernetes* probes and the **Horizontal Pod Autoscaler (HPA)** are the first line of defense to restore capacity automatically.

3 Requirements and SLOs

3.1 Functional Requirements

1. **High-Throughput Ingestion:** The system shall ingest internal bank transaction streams at rates up to 580 RPS, assigning unique identifiers to ensure full traceability across the distributed multi-cloud environment.
2. **Low-Latency Behavioral Lookup:** The system shall retrieve historical user profiles from internal databases via a Redis caching layer to identify behavioral anomalies without impacting real-time performance.
3. **Deterministic Resource Optimization:** The system shall utilize rule-based filters to instantly approve low-risk transactions, preserving AI agent resources for high-complexity investigations.
4. **Agentic Triage and Correlation:** The system shall deploy a Triage Agent to perform deep-dive investigations of suspicious alerts by correlating transactional data with infrastructure logs and telemetry via MCP-exposed tools.
5. **Supervisory Reasoning Audit:** The system shall utilize a Judge Agent to audit the Triage Agent's findings against bank policies and regulatory constraints, ensuring a consensus-based decision process.
6. **Secure-by-Design Tool Access:** The system shall enforce a hard security boundary via the Model Context Protocol (MCP), confining all agent actions to a pre-validated set of read/write tools without direct shell access.
7. **Human-in-the-Loop Escalation:** The system shall mandate human approval for any high-impact action, such as freezing high-value accounts, whenever agents cannot reach a high-confidence consensus.

3.2 Non Functional Requirements

1. **Scalability:** SharpWing manages internal transaction surges up to 580 RPS by dynamically scaling its microservices using horizontal pod autoscaling and event-driven architecture.
2. **Availability & Reliability:** SharpWing ensures 99.99% uptime for core banking functions through multi-zone deployments, effectively removing any single point of failure.
3. **Responsiveness:** SharpWing maintains sub-500ms (p95) scoring latency, for standard, non-escalated transactions, preventing fraud detection checks from impacting the speed of real-time customer payments.
4. **Security:** SharpWing protects sensitive financial data using Zero Trust principles and sandboxed MCP environments to enforce the least privilege for all agent operations.

5. **Automation Safety:** SharpWing enforces Human-in-the-loop approval for critical actions and imposes reasoning step limits to prevent unsafe or infinite agent loops.
6. **Observability:** SharpWing offers complete transparency through integrated logging and metrics, enabling rapid diagnosis of operational anomalies via centralized dashboards.
7. **Recovery:** SharpWing targets a Microservice MTTR (Mean Time To Repair) under 60 seconds, utilizing self-healing mechanisms and message buffering to eliminate data loss during system recoveries.
8. **Graceful Degradation:** SharpWing preserves essential payment processing using deterministic fallback rules whenever the AI layer or primary data stores are degraded.
9. **Budgetary Compliance:** SharpWing adheres to strict monthly budget limits for cloud and AI resources, ensuring all automated scaling remains cost-effective for the bank.
10. **Auditability:** SharpWing maintains immutable logs of all agent decisions and system changes, ensuring full accountability for DORA compliance and forensic analysis.

3.3 Compliance and Governance Requirements

1. **Data Sovereignty:** SharpWing enforces strict data residency by confining PII (Personally Identifiable Information) within regional boundaries (EU-only) through isolated regional clusters and encrypted cross-region replication.
2. **Explainability:** SharpWing provides human-readable justifications for every high-risk AI decision, ensuring compliance with AI Act transparency requirements for fraud detection systems.
3. **Identity-based Least Privilege:** SharpWing implements IAM Roles for Service Accounts (IRSA) and MCP-based sandboxing, ensuring agents have no direct shell access and operate with the minimum necessary API permissions.
4. **Business Continuity (DR):** SharpWing implements a Pilot Light disaster recovery strategy across multiple cloud regions, targeting a regional failover RTO of under 5 minutes to satisfy DORA resilience mandates.
5. **Economic Guardrails:** SharpWing implements a multi-tiered defense where deterministic and ML layers resolve over 99% of transactions, shielding the Agentic Layer from high-cost token consumption. Strict daily LLM quotas act as a circuit breaker against Economic Denial of Service (EDoS) attacks targeting the cognitive layer.

3.4 Service Level Objectives (SLO)

1. **Service Availability:** SharpWing targets a **99.99% monthly uptime** for core banking services, ensuring that the API Gateway and Scoring System are always reachable for real-time transactions.
2. **API Ingestion Latency:** SharpWing maintains a **p95 threshold < 50ms** for the synchronous ingestion phase, providing near-instant acknowledgement to the bank's payment terminals.

3. **Transaction Processing Latency:** SharpWing ensures an end-to-end asynchronous processing time (from ingestion to final verdict) with a **p95 threshold < 500ms** under nominal load.
4. **Recovery Time Objective (RTO):** SharpWing targets a recovery time of **< 60s** for automatic service restoration after a pod failure, utilizing Kubernetes self-healing.
5. **Recovery Point Objective (RPO):** SharpWing maintains a data loss window of **< 15s** by leveraging Kafka's persistent topic logs and real-time database replication.
6. **Agent Escalation Delay:** SharpWing enforces a maximum threshold of **2 minutes** for the Guardian Agent to resolve an infrastructure anomaly before escalating the incident to a human operator.
7. **Message Queue Processing Latency:** SharpWing monitors the Kafka backlog to ensure that the time messages spend in the queue stays within a **p95 threshold < 5s**, preventing permanent processing lag.
8. **Operational Data Freshness:** SharpWing ensures that observability metrics and transaction logs are updated and visible on dashboards with a **maximum delay of 15s**.
9. **Error Rate for Valid Requests:** SharpWing maintains a **threshold < 0.1%** for failed requests caused by internal server errors, ensuring high reliability for legitimate financial operations.
10. **Budget Compliance Alerting:** SharpWing triggers an immediate escalation if the daily LLM inference or cloud scaling costs exceed the predefined budget threshold by **more than 10%**.

4 System Architecture

The architecture of SharpWing is designed as a distributed, event-driven ecosystem of microservices, organized into a pipeline that functions as a sequence of increasingly refined filters for detecting fraudulent transactions.

4.1 Transactional Data Pipeline

The data flow within SharpWing is designed for high-throughput ingestion and low-latency processing, utilizing a series of specialized Kafka topics to manage state transitions:

- **Ingestion:** The process begins at the API Gateway, which performs syntactic validation and assigns a unique identifier to each transaction. The raw payload is then published to the raw-transactions topic.
- **Algorithmic Check:** This first level of filtering applies deterministic, rule-based logic to the transaction stream. It acts as an edge filter by instantly identifying known fraudulent patterns or blocklisted entities. By resolving high-confidence cases through simple algorithms, SharpWing prevents the over-provisioning of expensive compute resources for standard operations.
- **Machine Learning Check:** Transactions that pass the initial rules are consumed for probabilistic analysis. This module utilizes Machine Learning models to evaluate behavioral anomalies by cross-referencing historical user profiles retrieved from a Redis cache. The result is a granular risk score that determines if the transaction is clearly legitimate or requires expert investigation.
- **AI Agents Check:** The final and most sophisticated filter is activated only when the previous layers encounter high uncertainty or ambiguous patterns. The Triage Agent performs a deep-dive investigation using MCP-exposed tools to correlate telemetry and logs, while the Judge Agent audits the reasoning to ensure policy compliance. This stage represents the finer filter that provides an explainable verdict or escalates the decision to a human operator.

4.2 Persistency and Storage

The system separates immediate operational needs from long-term archival requirements through a tiered data layer:

- **Multi-Layer Redis Caching:** To ensure strict fault isolation and sub-millisecond responsiveness, each of the three operational layers manages its own dedicated Redis in-memory cache. This decentralized approach allows for the storage of geolocation metadata, historical behavioral profiles, and intermediate agentic context respectively, effectively eliminating redundant external API calls and preventing bottlenecks in the primary data store.
- **TimescaleDB (Single Source of Truth):** Acts as the definitive archive of transactions.

- **Kafka Connect:** To avoid disk I/O bottlenecks during peak traffic, database writes are offloaded to Kafka Connect. This module persists verdicts in the background without affecting the real-time pipeline.

4.3 Architectural Trade-Offs Analysis

The design of SharpWing involves deliberate engineering compromises to prioritize availability and scalability:

- **Persistent Buffering/Transactional Latency:** The architecture utilizes Apache Kafka for persistent spike buffering to ensure that no transaction is lost during a component failure, though it introduces a model of eventual consistency where the final archival is the relational database.
- **Fault Isolation/Infrastructure Footprint:** The design utilizes independent microservices and decentralized data layers (Redis caches) to ensure strict fault isolation. This ensures that a failure in the enrichment stage does not paralyze the ingestion gateway. The trade-off is a significantly larger resource footprint and increased operational complexity, which is mitigated through mandatory distributed tracing via a unique identifier.
- **Elastic Scaling/Provisioning Predictability:** Relying on the Horizontal Pod Autoscaler for dynamic replication optimizes the Total Cost of Ownership by reducing idle resources. However, this introduces a trade-off regarding cold-start latency during sudden spikes. While static over-provisioning would eliminate this lag, SharpWing uses the message broker to absorb surges while the cluster scales out.
- **Deterministic Filtering/Analytical Precision:** To maintain service availability under heavy load and prevent uncontrolled LLM expenditure, the system implements deterministic edge filtering. This follows a graceful degradation strategy: the system trades a perfect agentic analysis for a good-enough deterministic verdict for standard transactions, preventing the AI agents from being overwhelmed by high-volume, low-complexity traffic.
- **Agentic Agreement/Computational Efficiency:** The system employs a multi-agent supervision model (Triage and Judge) to reach an agreement on high-uncertainty alerts. While this mechanism drastically reduces the risk of LLM hallucinations and unsafe operational proposals, it increases the token consumption and API request rates. This design choice prioritizes operational safety and decision integrity over the minimization of inference costs. To prevent uncontrolled expenditure during traffic spikes, the system implements cognitive load shedding, falling back to ML-based verdicts if the daily LLM token budget is exhausted.

5 Reliability and Scalability

5.1 Importance of Scalability and Reliability

For a mission-critical internal platform operating on such an extensive scale, scalability and reliability represent crucial and indispensable properties. Without these foundational pillars, there is no viable path toward achieving the strategic objectives of securing the institution's real-time banking operations.

In the development of SharpWing, these requirements are prioritized equally with core functional features. This parity is justified both by the high level of engineering complexity involved in managing distributed state and by the decisive impact these properties have on the final outcome.

Ultimately, a service of this importance is not defined solely by its analytical capabilities, but by its ability to remain available and responsive under any operational condition; without absolute reliability, the core functionality itself loses its institutional value.

5.2 Scalability Mechanisms

To handle the variable workload and prevent bottlenecks SharpWing implements several horizontal and vertical scaling strategies:

- **Kafka-based Spike Buffering:** The system utilizes Apache Kafka to decouple the synchronous API ingestion from the resource-intensive analytical stages. During traffic bursts, Kafka acts as a buffer, safely queuing transactions and allowing the system to trade temporary processing latency for absolute availability.
- **Horizontal Pod Autoscaling (HPA):** Every microservice is containerized and managed by Kubernetes. The HPA monitors CPU and memory utilization, dynamically replicating pods to match the incoming demand. This ensures that the Scoring System and Agentic Workers can scale out linearly during peak hours.
- **Redis Profile Caching:** To avoid saturating the primary TimescaleDB database, user behavioral profiles are mirrored in a Redis cache. This allows the Scoring System to perform historical lookups with sub-millisecond latency, which is essential for maintaining a p95 processing SLO under 500ms.
- **Microservices Decoupling:** By isolating the different modules, the platform can scale individual components independently based on their specific resource requirements.

5.3 Reliability Mechanisms

The platform's resilience is built on a "Secure-by-Design" philosophy strongly radicated in infrastructural self-healing:

- **Multi-Cloud Active/Active Strategy:** SharpWing is deployed across **AWS and Azure** regions. In the event of a total regional outage or cloud provider failure, traffic is rerouted via DNS health checks, ensuring a Recovery Time Objective (RTO) of less than 5 minutes.
- **Kubernetes Self-Healing:** Liveness and Readiness probes continuously monitor pod health. Kubernetes automatically terminates and restarts instances in a Crash Loop, reducing the need for manual intervention during standard operational failures.
- **Hard Security Boundaries (MCP):** All agentic actions are confined within a Model Context Protocol (MCP) server. This prevents agents from executing arbitrary shell commands, ensuring that automated remediation is always safe, auditable, and compliant with the bank's internal policies.
- **Human-in-the-Loop (HITL):** For high-impact or ambiguous scenarios, the system implements a mandatory escalation policy. If the Judge Agent cannot validate a proposed action with high confidence, the decision is deferred to a senior bank operator.

6 Observability

6.1 Importance of Observability

In SharpWing, telemetry serves as the primary data source for the Triage Agent's reasoning and for the continuous verification of Service Level Objectives (SLOs). The system requires high-granularity data to validate cluster behavior during peaks and to ensure the auditability mandated by DORA and the AI Act.

6.2 Observability Mechanisms

SharpWing utilizes a modern, integrated observability stack designed for high-cardinality data and low-latency querying across its multi-cloud Kubernetes environment:

- **Prometheus (Metrics):** Acts as the primary time-series database. It operates in a "pull" mode, scraping metrics from microservice endpoints, the Kafka cluster, and Kubernetes nodes. It provides the quantitative data required by the Horizontal Pod Autoscaler (HPA).
- **Promtail and Loki (Logs):** For log aggregation, Promtail is deployed as a DaemonSet on every node to ship container logs to a centralized Loki instance. Unlike traditional indexing, Loki indexes only the metadata (labels), allowing for high-velocity ingestion.
- **Grafana (Visualization):** Serves as the unified single-pane-of-glass for the SOC and SRE teams. It integrates both Prometheus and Loki data sources, enabling operators to move from a metric anomaly (e.g., a spike in latency) to the corresponding log entries in a single click, facilitating rapid incident response.

6.3 Metrics

The metrics collected are categorized into infrastructure and application levels to ensure a holistic view of the system's health and efficiency:

6.3.1 Infrastructure Metrics

- **Pod Resource Utilization:** Monitoring CPU and Memory requests versus limits to detect resource starvation or Pod Crash Loops.
- **Kafka Consumer Lag:** The number of unprocessed messages in the raw-transactions topic. This is the primary indicator of whether the system is successfully performing spike buffering or if the scoring engine is falling behind.
- **Database Connection Pool Health:** Monitoring active versus idle connections in TimescaleDB and Redis to prevent I/O bottlenecks.

6.3.2 Application Metrics

- **Transaction Throughput (RPS):** Real-time measurement of incoming requests.
- **Ingestion Latency (p95):** Measuring the time between the API Gateway receiving a request and the message being published to Kafka. The target is less than 50ms.
- **Agent Token Consumption:** Granular tracking of LLM usage per incident to enforce the Agent Cost Ceiling and monitor the ROA.
- **Fraud Scoring Error Rate:** Percentage of transactions resulting in a "5xx" error or a "failed" verdict due to technical timeouts or agent reasoning loops.
- **Human-in-the-Loop Escalation Rate:** Frequency of incidents that required manual intervention, used to tune the Judge Agent's confidence thresholds.

7 Anomaly Detection

7.1 Adversarial Fraud Anomaly Typologies

When an adversarial persona is triggered by the ingestion loop, the generator marks the transaction flag `is_fraud = True` and injects one of four explicit behavioral anomalies:

- **Stolen Card Anomaly (Fraud_StolenCard):**

$$A_{\text{stolen}} \in [\$500.00, \$2,000.00]$$

Simulates a sudden, high-burst transaction velocity profile executed from an un-baselined device. The geospatial vector is dynamically pulled out of unknown high-risk corridors outside the user's home boxes, representing immediate credential theft.

- **Financial Smurfing Anomaly (Fraud_Smurfing):**

$$A_{\text{smurfing}} \in [\$9,900.00, \$9,999.00]$$

An explicit compliance evasion attack vector. The amounts are carefully restricted to a tight boundary hovering immediately below the standard \$10,000.00 Anti-Money Laundering (AML) institutional reporting threshold to test whether the deterministic tripwires correctly catch structural patterns designed to dodge regulatory detection.

- **Account Takeover Anomaly (Fraud_ATO):**

$$A_{\text{ato}} \in [\$20,000.00, \$50,000.00]$$

Simulates malicious account compromise followed by a massive, high-volume wallet drain to an unverified recipient. Because this vector generates high value spikes, it bypasses standard retail consumer ranges, forcing heavy reliance on historical user baselines.

- **Impossible Travel Geospatial Anomaly (Fraud_ImpossibleTravel):**

$$A_{\text{travel}} \in [\$5,000.00, \$15,000.00]$$

Forces a direct temporal-spatial conflict in the telemetry data. The generator actively relocates the transaction coordinates to one of four high-risk global regions (Southeast Asia, Africa, Eastern Europe, or South America):

$$\text{Geo}_{\text{high_risk}} \in \begin{cases} \text{Lat} \in [-10.0, 20.0], \text{Lon} \in [95.0, 140.0] & (\text{Southeast Asia}) \\ \text{Lat} \in [-35.0, 35.0], \text{Lon} \in [-17.0, 51.0] & (\text{Africa}) \\ \text{Lat} \in [40.0, 60.0], \text{Lon} \in [20.0, 50.0] & (\text{Eastern Europe}) \\ \text{Lat} \in [-55.0, 12.0], \text{Lon} \in [-80.0, -35.0] & (\text{South America}) \end{cases}$$

This pattern forces an immediate violation of physical constraints relative to the user's recent in-memory database telemetry, explicitly testing the multi-agent travel evaluation tools.

7.2 Architecture: Hybrid Anomaly Detection Engine

The first phase of anomaly detection system serves as the frontline, high-throughput filtering mechanism within the transaction pipeline. To balance microsecond-lane latency with deep predictive accuracy, the service architecture combines an in-memory *Deterministic Rule Engine* with an adaptive, probabilistic *Machine Learning (ML) Inference Engine*. This dual-layer structure operates as a high-efficiency sieve gate, automating clear-cut decisions while isolating ambiguous anomalies for downstream multi-agent evaluation.

7.2.1 Deterministic Rule Engine and In-Memory State Tracking

Before invoking heavy machine learning model matrices, all inbound transaction vectors are subjected to real-time structural and behavioral tripwires. This layer tracks states across rolling windows to catch immediate fraud signatures:

- **Financial Smurfing Intercept:** Evaluates transaction amounts to catch compliance evasion patterns designed to sit immediately below the standard regulatory threshold:

$$\text{Tripwire}_{\text{smurf}} = \text{True} \iff \$9,900.00 \leq A \leq \$9,999.00$$

- **Account Takeover (ATO) Window Filter:** Flags high-volume asset drains executed during historical high-risk hours ($H_{\text{tx}} \leq 4 \vee H_{\text{tx}} \geq 23$):

$$\text{Tripwire}_{\text{ato_window}} = \text{True} \iff A \geq \$20,000.00 \wedge H_{\text{tx}} \notin [5, 22]$$

- **Geospatial Impossible Velocity Vectoring:** Tracks spatial-temporal continuity. The module queries the client's last cached position and timestamp from the Redis store via a 24-hour key. It applies the Haversine formula to compute the physical distance d between coordinates. Let Δt be the elapsed time since the last recorded transaction. An anomaly is flagged if the implied travel speed exceeds physical commercial boundaries:

$$V_{\text{implied}} = \frac{d}{\Delta t} > 1000 \text{ km/h}$$

- **High-Frequency Burst Counter:** Enforces an operational rate-limit on individual accounts using atomic Redis increments (INCR). If an account exceeds 4 transaction attempts within a sliding 60-second window, it triggers an instant velocity block:

$$\text{Tripwire}_{\text{burst}} = \text{True} \iff \text{Hits}_{60\text{s}} > 4$$

If any deterministic rule is breached, the engine short-circuits the pipeline, appends the specific violation metadata to the transaction payload, and pushes the record directly to the related Kafka topic for immediate agentic triage, bypassing the local ML module entirely.

7.2.2 Probabilistic Sieve Gates and Machine Learning Inference

Transactions that clear the deterministic filters pass directly into the machine learning inference module. The core predictive core relies on an `XGBClassifier` that outputs a calibrated fraud probability score $S_{\text{risk}} \in [0, 1]$ based on the transaction amount, latitude, and longitude.

Mathematical Formalization of Probability Estimation

Fundamentally, the gradient boosted tree ensemble performs non-linear classification by summing the margin outputs of K constrained, shallow regression trees (where $K = 100$ and $\text{max_depth} = 3$). For a given transaction feature vector $\mathbf{x}_i = [A_i, \text{Lat}_i, \text{Lon}_i]^T$, the raw aggregate prediction margin $F(\mathbf{x}_i)$ is computed as the cumulative sum of scores assigned by individual base learners f_k :

$$F(\mathbf{x}_i) = \sum_{k=1}^K f_k(\mathbf{x}_i) \quad \text{where } F(\mathbf{x}_i) \in (-\infty, +\infty)$$

When calling the inference wrapper `predict_proba()`, the model transforms this unconstrained continuous real margin into a calibrated, bounded conditional probability score. This is mathematically achieved by passing $F(\mathbf{x}_i)$ through the standard logistic sigmoid activation function, derived directly from the principle of Maximum Likelihood Estimation (MLE) under a Bernoulli distribution:

$$S_{\text{risk}} = P(y_i = 1 \mid \mathbf{x}_i) = \sigma(F(\mathbf{x}_i)) = \frac{1}{1 + e^{-F(\mathbf{x}_i)}}$$

During the optimization phase, the parameter matrices are updated by minimizing the negative binomial log-likelihood loss function $\mathcal{L}$ regularized over tree complexities $\Omega(f_k)$:

$$\mathcal{L} = - \sum_{i=1}^N [y_i \ln(S_{\text{risk}}) + (1 - y_i) \ln(1 - S_{\text{risk}})] + \sum_{k=1}^K \Omega(f_k)$$

where $y_i \in \{0, 1\}$ represents the ground-truth fraud label. This mathematical framework ensures that the continuous score processed by downstream routing mechanisms reflects a valid, statistically calibrated likelihood metric rather than an arbitrary distance value.

To maximize operational throughput and minimize downstream Large Language Model token expenses, the system maps S_{risk} to three hardcoded MLOps sieve gates:

$$\text{Sieve_Gate}(S_{\text{risk}}) = \begin{cases} \text{Auto-Approval} & S_{\text{risk}} \leq 0.15 & \rightarrow \text{Route to Settlement} \\ \text{Grey Zone Triage} & 0.15 < S_{\text{risk}} < 0.80 & \rightarrow \text{Route to Agent Queue} \\ \text{Auto-Denial} & S_{\text{risk}} \geq 0.80 & \rightarrow \text{Route to Intercept Logs} \end{cases}$$

By resolving high-confidence safe transactions (≤ 0.15) and definitive fraud patterns (≥ 0.80) locally, the system restricts expensive agentic cognitive processing exclusively to ambiguous edge-cases falling in the *Grey Zone*.

Empirical Performance Evaluation

To validate the statistical stability and generalization performance of the baseline `XGBClassifier`, the module executes an automated performance evaluation cycle on a completely isolated validation split. Out of the initial 3,000 captured transactions, the dataset is stratified into a Training cluster ($N_{\text{train}} = 2100$), a Validation cluster ($N_{\text{val}} = 450$), and a completely Unseen Test Set ($N_{\text{test}} = 450$).

The empirical classification metrics recorded on the unseen evaluation partition are formalized in Table 7.1.

An analysis of the empirical distribution yields crucial system properties:

Table 7.1: XGBClassifier Performance Evaluation Matrix on Unseen Test Set

Class Indicator	Precision	Recall	F1-Score	Support
Safe (Benign Baselines)	0.98	0.97	0.97	395
Fraud (Adversarial Anomaly)	0.79	0.87	0.83	55
Accuracy			0.96	450
Macro Average	0.88	0.92	0.90	450
Weighted Average	0.96	0.96	0.96	450

- **High-Security Recall Corridor (0.87):** The model successfully flags 87% of all active, adversarial fraud vectors hidden in the unseen pipeline. This high sensitivity minimises the risk of malicious transactions leaking into the financial settlement layer without inspection.
- **Precision Calibration (0.79):** Out of the total alarms raised by the classifier, 79% represent actual fraud personas. The remaining 21% false-alarm rate represents healthy ambiguity, which validates the architectural necessity of routing these edge cases directly to the multi-agent cognitive triage layer.
- **Macro F1 Equilibrium (0.90):** The high macro average scores prove that despite severe class imbalance inside the sample stream (395 safe instances vs. 55 fraud instances), the tree splitting weights prevent model degradation or single-class bias.

Upon successful metric convergence, the finalized tree matrix parameters are serialized directly to disk to serve as the live active inference control.

7.2.3 Self-Bootstrapping and Data Diversity Guard

When the infrastructure cluster is launched from a cold start, the machine learning inference engine faces an immediate structural dependency: the localized `XGBClassifier` cannot initialize without pre-existing serialized model artifacts on disk (`fraud_model.pkl`). To decouple service initialization from manual developer intervention, the system implements an automated synchronization guard via a specialized bootstrap lifecycle manager.

$$\text{System State} = \begin{cases} \text{Blocking Sleep Loop} & \text{if } N_{\text{records}} \leq 500 \vee \max(A) \leq \$500.00 \\ \text{Execute Baseline Train} & \text{if } \text{FileExists}(\theta_{\text{champion}}) = \text{False} \wedge \text{Diversity} = \text{Passed} \\ \text{Handoff to Retrain Worker} & \text{if } \text{FileExists}(\theta_{\text{champion}}) = \text{True} \end{cases}$$

Figure 7.1: Bootstrap Initialization State Control

The initialization sequence executes in three deterministic phases:

1. **Data Diversity Verification:** On startup, the script opens an asynchronous connection pool to TimescaleDB and enters a blocking poll loop every 3 seconds. It executes a dual-aggregation query measuring the historical data volume injected by the data seeder microservice. The cluster blocks execution until the seeded dataset satisfies a minimum volumetric and variance profile envelope:

$$\text{Condition}_{\text{diversity}} = \text{True} \iff N_{\text{records}} > 500 \wedge \max(A) > \$500.00$$

This prevents the network from training a degenerate model on uniform placeholder data or single-class transaction values.

2. **Constrained Dataset Capping:** Once the database diversity check clears, the engine triggers the core training sequence. Although the historical seeder script injects an initial batch volume of 15,000 base transactions into TimescaleDB, the underlying training script explicitly caps its baseline consumption using an ordered random sample constraint:

$$\mathcal{D}_{\text{train}} \sim \text{Uniform}(\mathcal{D}_{\text{timescaledb}}) \quad \text{s.t.} \quad |\mathcal{D}_{\text{train}}| = 3000$$

By strictly sampling a subset of 3,000 historical transactions instead of memorizing the entire 15,000 seed batch, the architecture explicitly limits early-stage model memorization. This constraint prevents overfitting on static simulation data patterns and preserves mathematical generalization capacity for subsequent real-time streaming feedback cycles.

3. **Handoff Execution:** After the baseline training finishes, the serialized model matrix is saved directly onto the shared Docker storage volume mount point as the stable control artifact. With the baseline parameters secured, the bootstrap service executes a process handle handoff using the POSIX system call. This completely replaces the current execution thread memory space with the active, event-driven `ml_retrain_worker` loop, transitioning the system into its live deployment state with zero operational overhead.

7.2.4 Adaptive MLOps Lifecycle: Champion vs. Challenger Architecture

To prevent accuracy degradation caused by shifting real-world fraud patterns, anomaly detection employs an isolated *Champion vs. Challenger* deployment strategy.

- **Champion Model (θ_{champion}):** The baseline control model currently running the active, live inference gate logic and determining production routing rules.
- **Challenger Model ($\theta_{\text{challenger}}$):** A separate, continuously updated model matrix that processes live inputs in a parallel *Shadow Audit Mode* to evaluate alternative feature splits without impacting production routing.

7.2.5 Event-Driven Retraining Loop

The model evolution lifecycle is entirely event-driven and managed by an isolated background consumer service. To conserve local compute resources, the retrain container is resource-optimized, remaining idle until a specific operational threshold is met:

1. The worker subscribes to the human resolved topic of Kafka, listening for explicit manual resolutions published by human operators interacting with the Human-in-the-Loop (HITL) dashboard.
2. An internal counter tracks inbound resolutions. The exact moment a batch target of **10 freshly labeled records** (10/10) is reached, the retraining sequence activates.
3. The worker pulls up to 3,000 historical rows from TimescaleDB using an asynchronous connection. It restricts the dataset to finalized states, excluding raw receipts or open escalations to shield the model from memorizing simulation noise.
4. The data is split into stratified segments: 70% Training, 15% Validation, and 15% entirely Unseen Test sets. The training run enforces strict tree depth limits ($\text{depth} = 3$) and early stopping criteria to maximize broad generalization.

5. Upon a successful run, the newly optimized weights are serialized directly to disk as a fresh Challenger artifact (`challenger_model.pkl`).

To safely pull the fresh weights into production memory without causing downtime, an asynchronous file modification timestamp watcher polls the shared storage directory every 10 seconds. The moment it detects a fresh challenger file, it triggers a live hot-swap reload, updating the running application pointers instantly with zero message loss.

7.2.6 Architectural Graceful Degradation Framework

To ensure operational availability during microservice stress, cluster migration, or storage volume wipes, the first phase of anomaly detection implements a multi-tiered *Graceful Degradation* hierarchy:

$$\theta_{\text{active}} = \begin{cases} \theta_{\text{challenger}} & \text{if FileExists}(\theta_{\text{challenger}}) = \text{True} \wedge \text{Check}(\theta_{\text{challenger}}) = \text{Valid} \\ \theta_{\text{champion}} & \text{if FileExists}(\theta_{\text{challenger}}) = \text{False} \vee \text{Check}(\theta_{\text{challenger}}) = \text{Corrupted} \end{cases}$$

Figure 7.2: System Inference Pointer Fallback Matrix

1. **Inference Matrix Redundancy:** If the automated retraining loop generates a corrupted model file, or if the shared storage directory is wiped at cluster startup, the inference loader catches the fault natively. Instead of throwing a runtime crash exception, the system degrades gracefully by mirroring the stable control Champion matrix pointers into the Challenger slot.
2. **Cognitive Sieve Fallback:** If the upstream multi-agent layer hits external token rate-limits or experiences an infrastructure outage, the first phase of anomaly detection system completely decouples itself. The pipeline sheds the expensive agentic reasoning layer entirely and shifts processing loads onto the localized XGBoost sieve gates, ensuring that transaction throughput never stalls.

7.3 Data Persistence and Write-Isolation Framework

To maintain strict decoupled state synchronization across distributed microservices without inducing synchronous network bottlenecks, the system architecture implements an asynchronous database writer pattern modeled after Command Query Responsibility Segregation (CQRS) principles. Microservices executing runtime automated inferences, such as the `XGBClassifier` sieve gates, the multi-agent workflow engines, or the manual Human-in-the-Loop (HITL) dashboard API, are entirely decoupled from the relational database persistence layer.

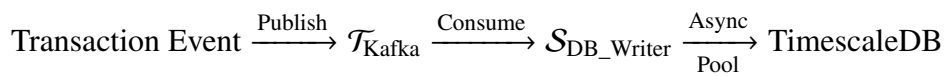


Figure 7.3: Asynchronous CQRS Write-Isolation Pipeline

Rather than opening direct, blocking relational database links to execute immediate SQL updates upon calculating a final fraud verdict, operational microservices act purely as upstream event producers. The finalized payload state is published asynchronously to dedicated transactional topics.

A specialized, downstream consumer service, the database writer or reporter module, is solely responsible for managing state mutations within the time-series database engine (TimescaleDB). This persistence architecture operates through three primary structural mechanisms:

- **Asynchronous Connection Pooling:** The persistence service maintains a highly scalable, non-blocking connection infrastructure. This optimization allows the service to consume, batch, and flush incoming Kafka event messages concurrently without locking database worker threads.
- **Idempotent State Mutations:** When a message is consumed from a finalized topic, the writer performs an idempotent upsert operation on the central transactional ledger. Let $\mathbf{t}$ be the inbound transaction vector and $\mathcal{R}$ be the target relational database relation. The mutation function γ maps the payload properties to an atomic write-or-update state schema:

$$\gamma(\mathbf{t}, \mathcal{R}) \rightarrow \mathcal{R} \cup \{\mathbf{t}_{\text{id}} \mapsto [\text{state}(\mathbf{t}), \text{is_fraud}(\mathbf{t}), \text{reason}(\mathbf{t})]\}$$

By structuring updates as a single relational collision-safe upsert based on the unique `transaction_id` primary key, the system guarantees absolute data consistency and idempotency across streaming channels, completely eliminating the risk of race conditions or duplicate record tracking.

- **Absolute Write-Isolation Security:** By routing all database operations through localized Kafka commit logs rather than standard transactional database access chains, the pipeline achieves absolute write isolation. In the event that the backend storage cluster experiences heavy row locking under adversarial stress or is temporarily taken offline for rolling maintenance partitions, the primary fraud evaluation lane remains completely unaffected. Upstream inference streams continue processing transactions at line rate, safely buffering final states in Kafka’s append-only logs until database connectivity is restored and the reporter service catches up automatically.

8 Agentic Operations Layer and Safety

The Agentic Operations Layer represents the intelligent orchestration core of SharpWing. Unlike traditional automated systems, this layer does not replace the human operator but provides advanced support through incident triage investigation and operational summarization, all governed by a rigorous safety framework.

8.1 Triage Agent

The **Triage Agent** is the system's primary investigative component, specifically designed to bridge the gap between deterministic automation and human intervention.

- **Activation Trigger:** The agent is dispatched when the core Machine Learning (ML) models encounter high-uncertainty scenarios or when suspicious patterns are detected but do not reach a predefined confidence threshold for autonomous action. Specifically, this occurs when XGBoost classifier of the first phase of anomaly detection system yields a risk score inside the high-ambiguity Grey Zone ($0.15 < \text{score} < 0.80$), or when a high-speed deterministic tripwire is explicitly breached, forcing an immediate async push to the respective Kafka stream.
- **Investigation via MCP:** Upon activation, the Triage Agent utilizes the Model Context Protocol (MCP) to access the telemetry and logging stack. It queries metrics from Prometheus and inspects logs in Loki to reconstruct the context of a potential fraud or infrastructure anomaly. In practice, the agent relies on an active stdio client connection to a FastMCP server, allowing it to dynamically execute specialized database and cache queries. This setup enables real-time interaction with the infrastructure stack via tools such as `get_user_history` to review transactional states, `evaluate_impossible_travel` to inspect spatial-temporal telemetry, and `evaluate_daily_velocity` to analyze 24-hour liquidity accumulations inside the TimescaleDB layer.
- **Output Generation:** Its primary goal is to perform "alert triage," correlating disparate data points to identify a root cause and proposing a specific remediation or verdict. The final output is structurally confined to a strict JSON format that provides the downstream pipeline with a boolean determination of fraud, an explicit float indicating risk confidence, a recommended action, and a transparent, tool-audited text explanation mapping out the reasoning behind the decision.

8.1.1 API Rate-Limit Resilience via Exponential Backoff

Because the agentic system cognitive layer relies on advanced Large Language Model (LLM) orchestration to evaluate high-ambiguity transactions in the Grey Zone, its runtime loops are highly dependent on external inference API availability. Under sudden bursts of streaming anomalies, concurrent multi-agent invocations can easily exhaust local API token buckets or trigger provider-side throttling, throwing an HTTP 429 (Too Many Requests) exception. To prevent these network bottlenecks from hard-crashing the microservice or dropping active Kafka messages mid-flight, the triage execution thread embeds an automated, non-blocking *Asynchronous Exponential Backoff and Retry* protocol. Let t_{sleep} be the calculated pause interval,

b represent the exponential base scaling factor, and i indicate the current failure attempt index. The mathematical model governing worker delay is formulated as:

$$t_{\text{sleep}}(i) = t_{\text{base}} \times b^i \quad \text{where } t_{\text{base}} = 15\text{s}, b = 2, i \in \{0, 1, \dots, N - 1\}$$

The resilience mechanism executes deterministically through the following processing steps:

- **Non-Blocking Interval Scaling:** Upon intercepting an API connection crash or rate limit error, the worker pauses thread execution via an asynchronous delay. This scales the waiting period sequentially across attempts ($15\text{s} \rightarrow 30\text{s} \rightarrow 60\text{s}$).
- **Memory Invariant Safety:** Because the delay is entirely asynchronous, the individual worker pod can sit in a paused state waiting for its token bucket to replenish without locking up the underlying application core or stalling the overall Kafka consumer partition pool.
- **Loop Defect Isolation:** The maximum retry boundary is explicitly restricted to $N = 5$. If a highly congested network environment completely exhausts all 5 attempts without achieving a successful API handshake, the block drops out of the retry loop, logs a critical traceback error, and purposefully raises the exception. This forces the message offset back into the Kafka commit logs so it can be picked up safely by alternative worker threads, preventing data loss.

8.1.2 MCP Server Infrastructure and Tool Taxonomy

To grant the agentic cognitive layer stateful introspection capabilities without violating the microservice boundary isolation, the architecture hosts a specialized FastMCP server instance. This server exposes a secured, asynchronous telemetry interface that the Triage Agent can query dynamically using standard JSON-RPC payloads over an exchange transport link.

Rather than allowing the LLM agents to execute arbitrary raw database transactions, the interface restricts agentic operations to four deterministic, optimized context-extraction functions.

Tool 1: Historical Baseline Introspection (`get_user_history`)

This function opens an asynchronous link to the central database to extract transactional baselines for a specific identifier.

- **Relational Target:** Extracts properties $T = [\text{id}, \text{amount}, \text{location}, \text{state}, \text{is_fraud}, \text{reason}]$ sorted via t_{ms} in descending order.
- **Analytical Objective:** It is primarily utilized by the agent to detect Account Takeover (ATO) or Stolen Card anomalies by comparing the inbound streaming payload traits against a historical behavior array. If no past records exist, the function explicitly yields a specialized status literal:

$$O_{\text{history}} = \{\text{"status": "no_history_found"}\}$$

This output string systematically triggers the Cold-Start Rule within the primary agent's prompt guardrails.

Tool 2: Spatial-Temporal Collision Evaluation (`evaluate_impossible_travel`)

This contextual function verifies spatial-temporal continuity by calculating geographical transitions between successive streaming events.

- **Mathematical Logic:** The function isolates the immediate chronological predecessor transaction t_{n-1} for an account. It extracts the past location string L_{n-1} and timestamp TS_{n-1} , and computes the temporal delta in minutes:

$$\Delta t_{\min} = \frac{TS_{\text{current}} - TS_{n-1}}{1000 \times 60}$$

- **Analytical Objective:** The tool packages the spatial attributes alongside the calculated $\Delta t_{\min}$ into a structured JSON payload. This serves as the computational basis for the Triage Agent to calculate physical velocity capabilities and flag impossible geospatial jumps.

Tool 3: Rolling Liquidity Accumulation Tracking (`evaluate_daily_velocity`)

This analytical function monitors 24-hour volume velocity metrics to identify structured evasion patterns designed to circumvent reporting boundaries.

- **Mathematical Logic:** Let t_{now} be the current transaction timestamp. The function establishes a dynamic lookback boundary window:

$$W_{\text{start}} = t_{\text{now}} - 86,400,000 \text{ ms}$$

It executes a dual-aggregation query over the time-series cluster to compute the transaction count (C) and total accumulated volume (V):

$$V = \sum_{i \in W} A_i \quad \text{where } A_i \text{ is the transaction amount}$$

- **Analytical Objective:** This provides the agent with an explicit financial summary of rolling card usage. It allows the cognitive reasoning layer to immediately intercept financial smurfing vectors trying to hide underneath the regulatory reporting thresholds.

Tool 4: Distributed Infrastructure Health Telemetry (`check_system_health`)

This observability component provides the cognitive layer with system infrastructure health awareness.

- **Execution Logic:** The tool attempts an explicit database handshake query, combined with an atomic ping request directed at the active Redis caching instance.
- **Analytical Objective:** It returns an active network health status matrix. If a tool call fails or returns blank context data due to network degradation, the agent calls this health check to differentiate between a high-risk cold-start fraud scenario and a generic backend database outage.

8.2 Judge Agent

The Judge Agent serves as a mandatory supervisory layer, auditing the Triage Agent's logic to prevent hallucinations and ensure policy alignment.

- **Reasoning Audit:** Rather than just checking the final output, the Judge Agent evaluates the Triage Agent's entire reasoning path. It checks whether the data retrieved via MCP justifies the proposed conclusion. Operating as an independent, zero-temperature LLM-as-a-Judge instance initialized with custom prompt criteria, it cross-references the raw transaction payload with the primary agent's tool logs to guarantee that conclusions logically align with historical baselines.
- **Explanatory Feedback:** The Judge provides a detailed explanation for its consensus or dissent, explicitly stating why it agrees or disagrees with the Triage Agent's findings. This evaluation is serialized as a structural output containing an integer reasoning grade from 1 to 5, alongside a comprehensive text critique that details any observed logic fallacies, baseline calibration errors, or procedural compliance missteps.
- **Operational Summarization:** For cases requiring human attention, the Judge Agent synthesizes the technical investigation into a concise summary, allowing human operators to understand complex incidents without manually parsing logs. By translating low-level system traces and tool telemetry data into a streamlined operational overview, this synthesis allows human analysts to immediately review and action the transaction from the React Human-in-the-Loop frontend dashboard.

8.3 Agents Responsibilities

The architecture enforces a strict separation of duties to maintain system integrity:

- **Primary Triage (Triage Agent):** Responsible for data correlation, log analysis, and initial alert investigation. Its focus is on "technical diagnosis". It leverages stateful microservice interfaces and real-time execution loops to extract contextual background evidence across external data pools and identify localized fraud typologies.
- **Validation and Summarization (Judge Agent):** Responsible for auditing the diagnostic logic, enforcing governance rules, and translating technical outcomes into operational summaries for human review. It acts as a cold regulatory firewall, strictly reviewing the primary agent's data interpretations to block hallucinations and enforce mandatory policy thresholds.
- **Consensus Mechanism:** If the Triage and Judge agents reach a consensus, the system proceeds according to the policy; however, if they disagree, the workflow is immediately escalated to a human operator. A consensus confirming auto-approval (≤ 0.30) or auto-denial (≥ 0.85) routes the payload directly to the final Kafka persistence layer. Conversely, a low grade (≤ 3) or a deliberate veto flag (`force_human_review = True`) from the Judge bypasses automated gates, immediately shunting the transaction to the related Kafka topic for analyst resolution.

8.4 Safety, Policies and Guardrail

SharpWing uses policies and guardrails to ensure safe operational automation.

8.4.1 Primary Triage Agent Invariant Policies

The behavioral envelope of the Primary Triage Agent is strictly governed by systemic prompts that restrict operational autonomy, preventing unverified transactions from skipping downstream human oversight.

- **Mandatory Contextual Interrogation:** The agent is strictly barred from rendering an automated decision without executing a complete multi-tool check via the Model Context Protocol (MCP). It must actively call `get_user_history`, `evaluate_daily_velocity`, and `evaluate_impossible_travel` using the underlying `sender_id`.
- **The Cold-Start Rule:** Let H_x be the historical transaction footprint of a given client account x . If the context extraction returns `status = 'no_history_found'`, the system enforces an absolute policy: an absence of data must never be interpreted as proof of innocence. The profile represents an immediate risk of New Account Fraud (NAF) or Stolen Identity.
- **Confidence Boundary Floor Constraint:** For any account operating within a cold-start state ($H_x = \emptyset$), the primary agent is strictly forbidden from assigning a fraud confidence score C below the human escalation floor:

$$C \geq 0.31 \quad \forall \{x \mid H_x = \emptyset\}$$

- **Downstream Routing Calibration Rubric:** The agent's output confidence score $C \in [0, 1]$ is systematically mapped to clear operational sieve gates:

$$\text{Routing}(C) = \begin{cases} \text{Automated Approval} & 0.00 \leq C \leq 0.30 \quad (\text{Requires } H_x \neq \emptyset) \\ \text{Human Review Queue (HITL)} & 0.31 \leq C \leq 0.84 \\ \text{Automated Denial} & 0.85 \leq C \leq 1.00 \end{cases}$$

8.4.2 High-Net-Worth (VIP) Isolation and Behavioral Volatility Policies

The system architecture accommodates corporate accounts and high-net-worth individuals (VIP personas) by implementing specialized runtime policy profiles. Because these accounts routinely execute high-value transactions that deviate significantly from standard consumer baselines, treating them under a uniform behavioral envelope would trigger high volumes of false-alarm escalations, causing operational drag.

To maximize throughput while preventing security blind spots, the system calibrates its detection filters dynamically based on an explicit profile tier classification $\tau_x \in \{\text{Standard}, \text{VIP}\}$.

$$\text{Threshold}_{\text{ATO}}(x) = \begin{cases} \$20,000.00 & \text{if } \tau_x = \text{Standard} \\ \$100,000.00 & \text{if } \tau_x = \text{VIP} \end{cases}$$

Figure 8.1: Dynamic Account Takeover Rule Scaling Matrix

The specialized VIP policy framework operates through three core system mechanisms:

- **Dynamic Limit Expansion:** For verified VIP accounts, the deterministic in-memory tripwires are dynamically scaled. The Account Takeover (ATO) window filter threshold is expanded from the standard \$20,000.00 boundary floor up to a high-capacity \$100,000.00 ceiling. This expansion accommodates legitimate high-volume asset movements without short-circuiting the ingestion pipeline under standard conditions.
- **Contextual Sieve Calibration:** The XGBClassifier remains fully active to monitor structural anomalies, but the downstream routing sieve gates are shifted. For accounts tagged with $\tau_x = \text{VIP}$, the Grey Zone entry barrier is relaxed linearly to accommodate higher underlying baseline variance:

$$\text{Sieve_Gate}(S_{\text{risk}} \mid \tau_x = \text{VIP}) \rightarrow \text{Grey Zone Triage} \iff 0.35 < S_{\text{risk}} < 0.85$$

This shift prevents benign, high-value corporate transfers from flooding the operational queue, optimizing analyst bandwidth.

- **Agentic Contextual Auditing:** While deterministic parameters are expanded, the operational stringency of the agentic cognitive layer is fully maintained. If a VIP transaction breaks even the expanded boundaries and routes to the Triage Agent, the agent’s multi-tool check execution is enforced with absolute parity. The Model Context Protocol (MCP) server hooks directly into historical time-series logs to verify if the massive transaction matches the corporate user’s past multi-month liquidity patterns or spatial-temporal trajectories.

By combining expanded frontline thresholds with an uncompromised multi-agent secondary audit loop, the architecture provides elite tier flexibility without opening high-value evasion windows for adversarial manipulation.

8.4.3 Auditor Judge Agent Guardrails (LLM-as-a-Judge)

To mitigate the risk of Large Language Model hallucinations, miscalibrations, or logical degradation, an independent, zero-temperature Auditor Judge Agent executes continuous runtime quality checks.

- **Logic and Quality Rubric Scoring:** The Judge grades the Primary Agent’s reasoning chain on a strict scalar range $G \in \{1, 2, 3, 4, 5\}$. Any analytical loop that contradicts its own findings, overlooks active anomaly detection system triggers, or violates the Cold-Start Rule is assigned a failing evaluation ($G \leq 3$).
- **Deterministic Human Escalation Veto Overrides:** The Auditor Judge possesses ultimate veto authority over automated approvals. It is hardcoded to force human intervention by switching the flag $\phi_{\text{veto}} \rightarrow \text{True}$ if any of the following critical grading criteria are met:

$$\phi_{\text{veto}} = \begin{cases} \text{True} & \text{if } G \leq 3 \\ \text{True} & \text{if primary reasoning exhibits data hallucination} \\ \text{True} & \text{if severe risk is treated as weak/dismissive} \\ \text{True} & \text{if Cold-Start Fallacy Violation is observed} \\ \text{False} & \text{otherwise} \end{cases}$$

where a Cold-Start Fallacy Violation occurs if the primary agent improperly interprets a completely blank history as proof of safety to dismiss an active tripwire alert.

8.4.4 Structural Payload Enforcements

Both cognitive agents operate under a structural syntax guardrail. The orchestration loop enforces deterministic output formats through system instructions requiring pure JSON serialization:

$$O_{\text{agent}} \in \mathbb{S}_{\text{JSON}} \setminus \mathbb{S}_{\text{markdown}}$$

By explicitly barring markdown backticks, conversational preambles, and loose text explanations outside the JSON object, the system blocks syntax-induced ingestion faults, allowing clean, automated serialization into Kafka data streams.

9 Threat Model and Security

9.1 Network Attacks

- **Volumetric DDoS:** Well-known attacks aimed at exhausting the network bandwidth of the internal API Gateway. Mitigation relies on a hardware Web Application Firewall (WAF) at the perimeter and Reverse Proxy before the real Servers.
- **Economic Denial of Service (EDoS):** Application-level attacks designed to trigger massive scale-out events, rapidly depleting the bank's cloud budget. SharpWing mitigates this through rate limiting at the gateway and the Judge Agent's ability to enforce load shedding when transaction patterns suggest a coordinated EDoS attempt rather than legitimate market volatility.

9.2 Classic Injection Attacks

- **Input Validation:** The API Gateway performs strict schema validation on all incoming JSON payloads, neutralizing malformed transactions that could exploit vulnerabilities in the downstream microservices.
- **Data Layer Protection:** To prevent SQL injection in the TimescaleDB single source of truth, all data access is mediated by typed SDKs and parameterized queries. Direct database access is strictly prohibited for the agentic layer; agents can only query data through the MCP-exposed tools, which enforce read-only constraints and parameter sanitization at the middleware level.

9.3 Modern Injection Attacks

- **Indirect Prompt Injection:** This threat involves the embedding of malicious natural language instructions within user-controlled transactional data, designed to manipulate the Triage Agent's reasoning and bypass fraud detection logic. SharpWing neutralizes this risk through its Model Context Protocol (MCP) architecture, which enforces a strict security boundary between the agent and the banking core.

9.4 Security Principles and Best Practices

- **Principle of Least Privilege (RBAC):** Every service and agent operates with a minimal Kubernetes Service Account. Destructive actions are blocked at the API Server level.
- **Human-in-the-Loop (HITL) Policy:** SharpWing mandates human intervention in ambiguous scenarios to balance processing speed, decision precision, and inference costs. This prevents speculative autonomous actions, ensuring that complex interventions are only executed under high-confidence consensus or direct human verification.
- **Immutable Auditability:** In compliance with DORA and the AI Act, every reasoning step and tool invocation is logged in an immutable trail. This ensures full accountability and facilitates post-mortem analysis of agent-driven interventions.

10 Cloud Migration

The migration of SharpWing to a cloud-native environment is a strategic initiative designed to decouple the core banking logic from physical infrastructure constraints. By adopting a replatforming approach, the system leverages managed services to ensure absolute service continuity, fulfilling the stringent availability and security requirements of the financial sector. The transition facilitates a distributed, multi-cloud topology that eliminates single points of failure and provides a transparent, auditable platform for agentic operations.

10.1 Compliance: DORA, AI Act, GDPR

SharpWing's architectural choices are strictly dictated by the evolving European regulatory landscape for financial services and artificial intelligence.

- **DORA (Digital Operational Resilience Act):** To satisfy DORA's mandates on ICT risk management and operational resilience, the platform implements a Secure-by-Design infrastructure. The requirement for continuous availability is met through a multi-region deployment. Every infrastructure modification and agent-driven remediation is recorded in an immutable audit trail, ensuring full accountability during regulatory inspections.
- **EU AI Act:** In accordance with the AI Act requirements for high-risk AI systems, SharpWing enforces strict transparency and human oversight. The Judge Agent acts as a supervisory layer, auditing the Triage Agent's reasoning paths against predefined safety guardrails. Furthermore, the Human-in-the-Loop (HITL) policy ensures that no irreversible or high-impact action is executed autonomously, maintaining the legal and ethical responsibility of the final decision within the human operator's domain.
- **GDPR and Data Sovereignty:** Data sovereignty and privacy are managed through a Cell-based Architecture. This design ensures that sensitive financial records and PII (Personally Identifiable Information) are processed and stored within specific geographical boundaries, aligning with GDPR requirements for data residency. The use of Model Context Protocol (MCP) servers further protects data privacy by enforcing semantic boundaries, ensuring that AI agents only access the minimum necessary subset of data required for a specific investigation.

10.2 Multi-Cloud Deployment

The platform adopts a Pilot Light multi-cloud strategy to balance maximum operational resilience with cost efficiency. While Amazon Web Services (AWS) serves as the primary active region handling the entire transactional workload, Microsoft Azure acts as a "warm" secondary environment.

This approach leverages high-level managed services—such as Amazon EKS, RDS, MSK, and ElastiCache—to maintain the primary pipeline's high performance. In the secondary Azure region, only the essential data replication and control plane components remain active, significantly reducing idle compute costs compared to an Active/Active setup. This multi-cloud synergy ensures that SharpWing remains ready-to-scale on a different provider, preventing vendor lock-in and providing a secure fallback without the financial burden of redundant high-capacity clusters.

10.3 Disaster Recovery Plan

The SharpWing Disaster Recovery (DR) strategy is engineered to maintain business continuity in the event of a total regional failure, targeting a Recovery Time Objective (RTO) of less than 5 minutes and a Recovery Point Objective (RPO) of less than 1 second.

The DR plan is articulated through the following technical components:

- **Pilot Light Standby:** A minimal footprint of critical services is maintained in a "warm" state within the secondary Azure region. This includes the core control plane and basic data replication services, ensuring that the infrastructure is ready to scale out instantly without the overhead of maintaining a full-capacity redundant cluster.
- **Data Replication and State Integrity:** To achieve the sub-second RPO, SharpWing utilizes Amazon Aurora Global Database for asynchronous storage-level replication between AWS and Azure, with a typical lag of less than one second. Similarly, user behavioral profiles are synchronized via cross-region Redis replication, ensuring that the secondary region always possesses the near-real-time context necessary for fraud scoring.
- **Automated Failover Orchestration:** Regional health is continuously monitored by Route 53 (or equivalent Global Accelerator health checks). Upon detection of a primary region collapse, traffic is automatically rerouted via DNS to the secondary Azure ingress.
- **Rapid Elastic Recovery:** As the diverted traffic hits the secondary region, the Horizontal Pod Autoscaler (HPA) detects the sudden load and triggers a massive scale-out of the Pilot Light microservices.

11 Economic Analysis: ROI and Return on Agent (ROA)

The economic sustainability of SharpWing is evaluated as an internal banking solution focusing on Cost Avoidance. Unlike commercial SaaS models, the value is generated by mitigating direct financial losses from sophisticated frauds and optimizing operational efficiency.

11.1 Financial Analysis and Fraud Impact

- **Complex Fraud Value:** Based on market data for sophisticated banking fraud (e.g., Account Takeover or high-value authorized push payment fraud), the average loss per incident is estimated at €3,500.
- **Agent Success Rate:** We estimate that the agentic layer successfully identifies and blocks 8 complex frauds per month that would otherwise bypass traditional detection systems.
- **Total Monthly Benefit:** $8 \text{ incidents} \times €3,500 = €28,000/\text{month}$.

11.2 CAPEX, OPEX, and TCO

11.2.1 Capital Expenditure (CAPEX)

The initial investment covers design, development, and MVP validation for a total CAPEX of €50,000:

- **Software Engineering:** 3 specialized engineers for 3 months at a total cost of €45,000.
- **Cloud Infrastructure & Validation:** Staging and testing environments estimated at €5,000.

11.2.2 Operational Expenditure (OPEX)

The monthly recurring costs to maintain the platform at scale total €12,100/month:

1. **Cloud Infrastructure:** Managed services (EKS, MSK, Redis) and observability stack: €5,000/month.
2. **Payroll Expenses:** SRE and DevSecOps staff for high-risk escalations: €7,000/month.
3. **Agent Inference:** LLM token consumption for surgical investigations: €100/month.

11.2.3 Total Cost of Ownership (TCO)

The TCO for the first year (12 months) is calculated as:

$$\text{TCO}_{1y} = \text{CAPEX} + (\text{OPEX} \times 12) = 50,000 + (12,100 \times 12) = €195,200$$

11.3 Return on Investment (ROI)

The ROI measures the overall profitability of the SharpWing project by comparing the total avoided losses to the TCO.

- **Annual Gross Savings:** €28,000 × 12 months = €336,000.
- **Annual Net Savings:** €336,000 – €195,200 = €140,800.

$$\text{ROI} = \frac{\text{Annual Gross Savings} - \text{TCO}_{1y}}{\text{TCO}_{1y}} \times 100$$

$$\text{ROI} = \frac{336,000 - 195,200}{195,200} \times 100 = \mathbf{72.13\%}$$

Payback Period: The point where the cumulative savings equal the initial investment.

$$\text{Payback} = \frac{\text{CAPEX}}{\text{Monthly Benefit} - \text{Monthly OPEX}} = \frac{50,000}{28,000 - 12,100} \approx \mathbf{3.14 \text{ months}}$$

11.4 Return on Agent (ROA)

The ROA isolates the specific efficiency of the agentic layer.

In SharpWing, the agents benefit is the detection of complex frauds.

- **Agentic OPEX:** Includes inference tokens (€100) and a proportional fraction of bot hosting (€45), totaling €145/month.
- **Agentic Benefit:** Direct value of blocked frauds: €28,000/month.

$$\text{ROA} = \frac{\text{Agentic Benefit} - \text{Agentic OPEX}}{\text{Agentic OPEX}} \times 100$$

$$\text{ROA} = \frac{28,000 - 145}{145} \times 100 = \mathbf{19,210.3\%}$$

This exceptionally high ROA demonstrates that the agentic layer is a massive value multiplier: the negligible cost of LLM tokens is offset by the prevention of even a single high-value fraudulent transaction.

12 Future Developments and Conclusion

12.1 Future Developments

The journey of **SharpWing** marks the beginning of a new era in autonomous financial oversight. To ensure long-term viability, future developments are categorized into strategic innovations and necessitated evolutions.

12.1.1 Strategic Enhancements

These improvements aim to maintain the platform's competitive edge:

- **Easier Dashboards:** Improve dashboard explanations for non-technical operators.
- **Wider Range of Incidents:** Evaluate the agent layer on a larger set of incident scenarios.
- **Zero-RPO Multi-Cloud:** Moving towards synchronous consistency between AWS and Azure to achieve a *Recovery Point Objective (RPO)* of zero for critical states.

12.1.2 Evolutionary Necessity

External shifts may transform certain enhancements into mandatory requirements:

- **Security & Fraud Evolution:** As financial crimes evolve, the **State of the Art in detection** must advance. Continuous learning for the Triage Agent will likely become a necessity to counter increasingly sophisticated attack patterns.
- **Agentic Advancements:** Rapid breakthroughs in AI will set new industry benchmarks. Deep agentic reasoning, currently an innovation, may become a standard requirement for banking operations and licenses.
- **Regulatory Hardening:** Future legal frameworks beyond **DORA** and the **AI Act** may mandate stricter multi-cloud redundancy and total transparency in AI-driven decisions, forcing SharpWing to adopt even more granular cloud-agnostic and explainable AI measures.

12.2 Conclusion

SharpWing demonstrates that an Agentic Operations Layer is essential for mission-critical finance. By merging Kubernetes reliability with the automated reasoning of the Triage and Judge agents, the system maintains a 99.99% SLO even under stress. Our multi-cloud deployment ensures DORA compliance, while strict MCP guardrails guarantee operational safety. Ultimately, the positive Return on Agent (ROA) proves that SharpWing is a resilient, value-driven infrastructure that continues to behave acceptably when things go wrong.

13 Bibliography

- [1] Regulation (EU) 2022/2554 of the European Parliament and of the Council of 14 December 2022 on digital operational resilience for the financial sector (DORA). *Official Journal of the European Union*, L 333/1. Available at: EUR-Lex.
- [2] Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (AI Act). *Official Journal of the European Union*, L 2024/1689. Available at: EUR-Lex.
- [3] Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (GDPR). *Official Journal of the European Union*, L 119/1. Available at: EUR-Lex.
- [4] Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on measures for a high common level of cybersecurity across the Union (NIS 2). *Official Journal of the European Union*, L 333/80. Available at: EUR-Lex.
- [5] Directive (EU) 2015/2366 of the European Parliament and of the Council of 25 November 2015 on payment services in the internal market (PSD2). *Official Journal of the European Union*, L 337/35. Available at: EUR-Lex.
- [6] European Banking Authority (EBA). Guidelines on ICT and security risk management (EBA/GL/2019/04). Available at: EBA Official Portal.
- [7] Banca d'Italia. Circular No. 285 of 17 December 2013 - Supervisory provisions for banks (Chapter on the information system). Available at: Banca d'Italia Official Portal.
- [8] Deep Learning for Credit Card Fraud Detection: A Review of Algorithms, Challenges, and Solutions. Available at: ResearchGate.
- [9] A Review of Artificial Intelligence for Financial Fraud Detection. *Applied Sciences*. Available at: MDPI.
- [10] Credit Card Fraud Detection Dataset (ULB). Available at: Kaggle.